

IAM

user-3 EC2-Admin View, Start, and Stop Amazon EC2 instances

Tasks:

Step 1: Create IAM Groups and Attach Policies

In the AWS Management Console, navigate to IAM > Groups > Create New Group.

1. Create the required IAM groups: S3-Support, EC2-Support, and EC2-Admin

Group Name Managed Policy to Attach

S3-Support AmazonS3ReadOnlyAccess

EC2-Support AmazonEC2ReadOnlyAccess

EC2-Admin AmazonEC2FullAccess

2. Attach appropriate managed policies to each group based on the permissions specified.

3. Create IAM users: user-1, user-2, and user-3

Navigate to IAM > Users > Create user.

1. Create user-1, user-2, and user-3.

2. Select AWS Management Console access.

3. Set a custom password or use an autogenerated one.

4. Do not add them to groups yet (we will do this in Step 5).

4. Implement Multi-Factor Authentication (MFA) for all users and enforce it using an IAM policy.

Create the MFA Enforcement Policy

By default, "enforcing" MFA requires a custom JSON policy that denies actions if the user is not signed in with MFA.

1. Go to IAM > Policies > Create Policy.

2. Click the JSON tab and paste a policy that uses the aws:MultiFactorAuthPresent condition.

3. Name it MFA-Enforcement-Policy.

Note: A standard enforcement policy effectively says: "Allow these actions only IF aws:MultiFactorAuthPresent is true; otherwise, Deny."

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "AllowUserToManageTheirOwnMFA",
      "Effect": "Allow",
      "Action": [
        "iam:CreateVirtualMFADevice",
        "iam:EnableMFADevice",
        "iam:ListVirtualMFADevices",
        "iam:ResyncMFADevice"
      ],

```

```

"Resource": [
  "arn:aws:iam::*:mfa/${aws:username}",
  "arn:aws:iam::*:user/${aws:username}"
],
{
  "Sid": "DenyEverythingIfNoMFA",
  "Effect": "Deny",
  "NotAction": [
    "iam:CreateVirtualMFADevice",
    "iam:EnableMFADevice",
    "iam:ListVirtualMFADevices",
    "sts:GetSessionToken"
  ],
  "Resource": "*",
  "Condition": {
    "BoolIfExists": { "aws:MultiFactorAuthPresent": "false" }
  }
}
]
}

```

5.Assign each user to the corresponding group.

Navigate to IAM > Groups and add the users to their respective buckets:

- S3-Support: Add user-1
- EC2-Support: Add user-2
- EC2-Admin: Add user-3
- All Groups: Attach the MFA-Enforcement-Policy (created in Step 3) to every group or directly to the users.

All Groups: Attach the MFA-Enforcement-Policy (created in Step 3) to every group or directly to the users

6.Verify that permissions are correctly applied by testing access levels.|

user-1

User Expected Result

user-1 Can view S3 buckets; cannot start/stop EC2.

user-2 Can view EC2 instances; cannot start/stop them.

user-3 Can View, Start, and Stop EC2 instances.

All

Users

If logged in without MFA, they should be denied access to everything (due to the enforcement policy).

EBS

Step 1: Login to AWS Console

Open browser.

Go to

AWS Management Console

Login with AWS account credentials.

Step 2: Launch an EC2 Instance

In AWS Console search bar, search for

Amazon EC2

Open EC2 Dashboard.

Click Launch Instance.

Enter instance name:

EBS-Practical

Choose AMI:

Amazon Linux 2

Choose Instance Type:

t2.micro (Free Tier)

Create or select Key Pair.

In Network Settings:

Allow SSH traffic.

Click Launch Instance.

Step 3: Create an EBS Volume (Storage Provisioning)

In EC2 Dashboard left panel:

Click Elastic Block Store → Volumes

Click Create Volume.

Configure:

Volume Type: General Purpose SSD (gp2/gp3)

Size: 5 GB

Availability Zone: Same as EC2 instance

Click Create Volume.

Step 4: Attach EBS Volume to EC2 Instance

Select created EBS volume.

Click:

Actions → Attach Volume

Select EC2 instance:

EBS-Practical

Device name:

`/dev/sdf`

Click Attach Volume.

Step 5: Connect to EC2 Instance

Go to Instances.

Select instance.

Click Connect.

Use SSH connection.

Example command:

`ssh -i key.pem ec2-user@Public-IP`

Step 6: Check Attached Volume

Run:

`lsblk`

You will see new volume like:

`xvdf`

Step 7: Format the EBS Volume

Run:

`sudo mkfs -t ext4 /dev/xvdf`

Step 8: Mount the Volume

Create directory:

`sudo mkdir /data`

Mount volume:

`sudo mount /dev/xvdf /data`

Verify:

`df -h`

Step 9: Store Sample Data

Create file:

`echo "AWS EBS Practical Data" | sudo tee /data/test.txt`

Check:

`cat /data/test.txt`

Step 10: Create EBS Snapshot (Backup)

Go to:

EC2 → Volumes
Select EBS volume.

Click:

Actions → Create Snapshot

Enter description:

Backup Snapshot
Click Create Snapshot.

This snapshot acts as backup storage.

Step 11: Delete Original Data (Simulate Data Loss)

Inside EC2:

```
sudo rm /data/test.txt
```

Verify:

```
ls /data
```

Step 12: Restore Data Using Snapshot (Data Recovery)
Create New Volume from Snapshot

Go to:

EC2 → Snapshots
Select snapshot.

Click:

Actions → Create Volume from Snapshot
Choose same Availability Zone.
Click Create Volume.

Step 13: Attach Restored Volume
Select new volume.

Click:

Actions → Attach Volume

Attach to same EC2 instance.

Device name:

/dev/sdg

Step 14: Mount Restored Volume

Inside EC2:

Create folder:

```
sudo mkdir /restore
```

Mount restored volume:

```
sudo mount /dev/xvdg /restore
```

Verify recovered data:

```
cat /restore/test.txt
```

Recovered output:

AWS EBS Practical Data

S3 FILE

Task

Step 1: Open Amazon S3

1. Log in to the AWS Management Console.
2. In the search bar at the top, type S3 and select it from the services list.

Step 2: Create a New S3 Bucket

1. Click the orange Create bucket button.
2. Bucket name: Enter a unique name (e.g., ccl-practical-yourname-2026). Bucket names must be globally unique across all of AWS.
3. Region: Keep the default (e.g., us-east-1).
4. Object Ownership: Keep "ACLs disabled" (recommended) or "ACLs enabled" if your lab specifically requires older access methods.
5. Block Public Access settings: For this specific lab (Task 4), you must uncheck the box that says "Block all public access."
 - o Note: You will have to check a box acknowledging that this makes the bucket potentially public.
6. Click Create bucket.

Step 3: Upload Files

1. Click on the name of your newly created bucket.
2. Click Upload -> Add files.
3. Select a text document and an image from your computer.
4. Click the orange Upload button at the bottom.
4. Configure public access permissions for uploaded files.

Step 4: Configure Public Access Permissions

By default, even if the bucket allows public access, individual files are private. To make them viewable via URL:

1. Bucket Policy (The easiest way):

o Go to the Permissions tab of your bucket.

o Scroll down to Bucket policy and click Edit.

o Paste this policy (replace YOUR-BUCKET-NAME with your actual bucket name):

JSON

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "PublicReadGetObject",
      "Effect": "Allow",
      "Principal": "*",
      "Action": "s3:GetObject",
      "Resource": "arn:aws:s3:::ccl-practical-yourname-2026/*"
    }
  ]
}
```

2. Click Save changes.

Step 5: Access and Verify via URL

1. Go back to the Objects tab.

2. Click on the name of one of the files you uploaded (e.g., image.jpg).

3. Find the Object URL (usually looks like

[https://bucketname.s3.amazonaws.com/filename](https://bucket-name.s3.amazonaws.com/filename)).

4. Copy the URL and paste it into a new Incognito/Private browser window.

5. If the file opens or downloads, you have successfully completed the task!

RDS

Step 1: Login to AWS Console

Open browser.

Visit

AWS Management Console

Login using AWS credentials.

Step 2: Open Amazon RDS Service

In AWS search bar type:

RDS

Open

Amazon RDS Dashboard.

Step 3: Create Database

Click:

Create Database

Choose Database Creation Method:

Standard Create

Select Engine Type:

MySQL

Select Version:

Default MySQL version

Step 4: Choose Template

Select:

Free Tier

This helps avoid extra charges.

Step 5: Configure Database Settings

DB Instance Identifier

Example:

studentdb

Master Username

admin

Master Password

Create strong password.

Example:

Admin@123

Step 6: Configure Instance

DB Instance Class:

db.t3.micro

Storage Type:

General Purpose SSD (gp2/gp3)

Allocated Storage:

20 GB

Step 7: Connectivity Configuration

Select:

Public Access → Yes

Choose default VPC.

Security Group:

Create new security group.

Allow MySQL port:

3306

Step 8: Additional Configuration

Initial Database Name

college

Backup

Enable automatic backups.

Backup retention:

7 days

Step 9: Create Database

Review settings.

Click:

Create Database

Database creation takes a few minutes.

Step 10: Check Database Status

Go to:

RDS → Databases

Wait until status becomes:

Available

Step 11: Copy Endpoint

Open created database.

Copy:

Endpoint

Example:

studentdb.xxxxxx.ap-south-1.rds.amazonaws.com

This endpoint is used to connect the database.

Step 12: Connect to RDS Database

Using MySQL client:

Install MySQL client if required.

Connection command:

```
mysql -h studentdb.xxxxxx.ap-south-1.rds.amazonaws.com -u admin -p
```

Enter password when prompted.

Step 13: Verify Database

Run:

```
SHOW DATABASES;
```

Output will display:

```
college
information_schema
mysql
performance_schema
```

Step 14: Create Table

Use database:

```
USE college;
```

Create table:

```
CREATE TABLE student(
  id INT PRIMARY KEY,
  name VARCHAR(50),
```

```
course VARCHAR(50)
);
```

Insert values:

```
INSERT INTO student VALUES
(1,'Rahul','BCA'),
(2,'Priya','MCA');
```

Display records:

```
SELECT * FROM student;
```

S3 HOSTING

Step 1 & 2: Create Bucket and Configure Hosting

1. Open S3 and click Create bucket.
2. Bucket name: Use something unique like my-static-site-pratham-64.
3. Region: Keep it as us-east-1.
4. Public Access: Uncheck "Block all public access" and acknowledge the warning.
5. Create bucket.
6. Click on your new bucket -> Properties tab.
7. Scroll to the very bottom to Static website hosting and click Edit.
8. Select Enable.
9. Index document: Type index.html.
10. Click Save changes.

Step 3: Prepare and Upload Files

On your computer, create a simple file named index.html using Notepad or VS Code:

HTML

```
<!DOCTYPE html>
<html>
<head>
  <title>My AWS Website</title>
</head>
<body>
  <h1>Welcome to my S3 Static Website</h1>
  <p>This site is hosted on Amazon S3!</p>
  
</body>
</html>
```

1. In the Objects tab of your bucket, click Upload.
2. Add your index.html and an image file (make sure the image name matches exactly what you wrote in the code, e.g., my-image.jpg).

3. Click Upload.

Step 4: Enable Public Access (Bucket Policy)

1. Go to the Permissions tab.
2. Scroll to Bucket policy and click Edit.
3. Paste this JSON (replace YOUR-BUCKET-NAME with your actual bucket name):

JSON

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "PublicReadGetObject",
      "Effect": "Allow",
      "Principal": "*",
      "Action": "s3:GetObject",
      "Resource": "arn:aws:s3:::YOUR-BUCKET-NAME/*"
    }
  ]
}
```

4. Click Save changes

Step 5, 6 & 7: Access and Verify

1. Go back to the Properties tab.
2. Scroll to the bottom under Static website hosting.
3. You will see a link called Bucket website endpoint. It looks like:
http://your-bucket-name.s3-website-us-east-1.amazonaws.com
4. Click the link.
5. Your website should load showing your heading, text, and the image.

TERRAFORM

Q9 This solution provides a step-by-step guide to implementing Infrastructure as Code (IaC) using Terraform to provision an AWS EC2 instance.

Step 1: Install and Configure Terraform

- Install: Download the Terraform binary for your operating system (Windows/Linux/macOS) from the official website and add it to your system's PATH.
- Verify: Open your terminal and run `terraform -v` to ensure it is installed correctly.
- AWS CLI: Install the AWS CLI and run `aws configure` to set your Access Key, Secret Key, and Default Region so Terraform can authenticate with your account.

Step 2: Set Up the Working Directory

- Create a new folder for your project (e.g., `ccl-terraform-lab`).
- Open your terminal/command prompt and navigate into this directory.

Step 3: Create the Terraform Configuration

Create a file named main.tf in your directory and add the following code to define the provider, security group, and EC2 instance:

Terraform

1. Configure the AWS Provider

```
provider "aws" {  
  region = "us-east-1"  
}
```

2. Create Security Group for SSH (22) and HTTP (80) [cite: 112]

```
resource "aws_security_group" "allow_web_ssh" {  
  name = "allow_web_ssh"  
  description = "Allow SSH and HTTP inbound traffic"  
  ingress {  
    from_port = 22  
    to_port = 22  
    protocol = "tcp"  
    cidr_blocks = ["0.0.0.0/0"] # For SSH access [cite: 112]  
  }  
  ingress {  
    from_port = 80  
    to_port = 80  
    protocol = "tcp"  
    cidr_blocks = ["0.0.0.0/0"] # For HTTP access [cite: 112]  
  }  
  egress {  
    from_port = 0  
    to_port = 0  
    protocol = "-1"  
    cidr_blocks = ["0.0.0.0/0"]  
  }  
}
```

3. Launch Amazon EC2 Instance [cite: 110]

```
resource "aws_instance" "my_ec2" {  
  ami = "ami-0c55b159cbf0" # Amazon Linux 2 AMI [cite: 111]  
  instance_type = "t2.micro" # Free Tier type [cite: 111]  
  vpc_security_group_ids = [aws_security_group.allow_web_ssh.id]  
  tags = {  
    Name = "Terraform-Lab-Instance"  
  }  
}
```

Step 4: Initialize and Provision

1. Initialize: Run terraform init to download the necessary AWS provider plugins.
2. Preview: Run terraform plan to see exactly what resources Terraform will create before making changes.

3. Apply: Run terraform apply. Type yes when prompted to provision the resources on AWS.

Step 5: Verification and Validation

- Console Check: Log in to your AWS Management Console and navigate to EC2 > Instances to verify your instance is "Running".
- SSH Access: Use your terminal to connect to the instance: `ssh -i "your-key.pem" ec2-user@<public-ip>` to validate the deployment.

Step 6: Clean Up

- To avoid unwanted charges, run `terraform destroy`.
- Type yes to confirm the removal of the EC2 instance and the security group.

CLOUDWATCH

Step 1: Login to AWS Console

Open browser.

Visit

AWS Management Console

Login using AWS account credentials.

Step 2: Launch EC2 Instance

Search for

Amazon EC2

Open EC2 Dashboard.

Click:

Launch Instance

Configure Instance

Name

CloudWatch-Server

AMI

Amazon Linux 2

Instance Type

t2.micro

Key Pair

Create or select existing key pair.

Network Settings

Allow SSH traffic.

Launch

Click:

Launch Instance

Step 3: Open Amazon CloudWatch

In AWS search bar search:

CloudWatch

Open

Amazon CloudWatch Dashboard.

Step 4: View EC2 Metrics

In left panel click:

Metrics

Select:

All Metrics

Choose:

EC2

Click:

Per-Instance Metrics

Step 5: Monitor Important Metrics

Select EC2 instance metrics such as:

CPU Utilization

Shows percentage of CPU usage.

Metric:

CPUUtilization

Disk Operations

Shows disk read operations.

Metric:

DiskReadOps

Network Traffic

Shows incoming and outgoing traffic.

Metrics:

NetworkIn
NetworkOut

Graphs for these metrics will appear automatically.

Step 6: Create CloudWatch Alarm

In CloudWatch left panel click:

Alarms

Click:

Create Alarm

Click:

Select Metric

Choose:

EC2 → Per-Instance Metrics

Select:

CPUUtilization

Click:

Select Metric

Step 7: Configure Alarm Condition

Threshold Type

Static

Condition

Greater than 70

Meaning:

Alarm triggers when CPU usage exceeds 70%.

Step 8: Configure Notification Action

Under Notifications select:

In Alarm
Create new SNS topic.

Example:

EC2-Alerts
Enter email address.

Example:

student@gmail.com

Click:

Create Topic
Check email inbox and confirm SNS subscription.
Step 9: Name the Alarm

Example:

HighCPUAlarm

Click:

Create Alarm
Step 10: Connect to EC2 Instance

Go to EC2 dashboard.

Select instance → Click:

Connect

Use SSH command:

ssh -i key.pem ec2-user@Public-IP
Step 11: Generate CPU Load

Run the following command to increase CPU utilization:

yes > /dev/null &

Run multiple times for higher CPU usage:

yes > /dev/null &

yes > /dev/null &

yes > /dev/null &

Step 12: Verify Alarm

Return to CloudWatch.

Open:

Alarms

Alarm state changes from:

OK → In Alarm

Email notification will be received through SNS.

Step 13: Stop CPU Load

Inside EC2 run:

pskill yes

CPU utilization returns to normal.

Alarm changes back to:

OK

BEANSTALK

Complete Step-by-Step Guide for AWS Elastic Beanstalk (PaaS)

Deployment

Follow these steps exactly to complete Task 11 of your practical exam.

1. Application Initialization

- Open Console: Navigate to Elastic Beanstalk in the AWS Management Console.
- Create App: Click Create application.
- Name: Enter CCL-PaaS-App.
- Environment Tier: Select Web server environment.
- Platform: Select Python (or Node.js/PHP) from the dropdown.
- Application Code: Select Sample application.
- Configuration: Choose Single instance (free tier eligible).

2. Service Access (IAM Roles)

- Service Role: Select aws-elasticbeanstalk-service-role from the dropdown.
- EC2 Instance Profile: Select aws-elasticbeanstalk-ec2-role from the dropdown.

Note: If these are missing, create them in the IAM console with the permissions AWSElasticBeanstalkWebTier and AWSElasticBeanstalkManagedUpdatesC

ustomerRolePolicy.

- EC2 Key Pair: Select your key (e.g., cal).

3. Networking and Subnets

- VPC: Select the Default VPC (172.31.0.0/16).
- Instance Subnets: Check the boxes for all available Availability Zones (e.g., us-east-1a, us-east-1b) to ensure successful provisioning.
- Public IP: Ensure Activated is checked so the instance is accessible via the internet.

4. Instance and Scaling

- Instance Type: Verify it is set to t2.micro (within Free Tier limits).
- Scaling: Keep as Single Instance for the lab to avoid unnecessary resource creation.

5. Deployment and Verification

- Review: Click Skip to Review and then click Submit.
- Wait: The process takes 3–5 minutes. Monitor the events log for "Environment health is Green".
- Access: Click the Domain URL at the top of the dashboard to open your live sample website.

6. Post-Lab Management

- Health Check: Click the Health tab to show the examiner real-time status.
- Cleanup: Click Actions > Terminate Environment to delete all resources once you have finished your viva

HOST

Since you have your CCL-Lab-Instance already running, you can actually use it to complete this final task. You've already done the hard part of connecting to the terminal; now you just need to turn it into a web server and handle the resizing.

Step 1: Use Your Existing Instance

You are already using an Amazon Linux instance (i-06d9cb787f58b3a55). Make sure it is in the Running state.

Step 2 & 3: Install and Start Apache

Connect to your instance via EC2 Instance Connect (the terminal you used for the stress test) and run these commands one by one:

Bash

```
# Update the package manager
```

```
sudo yum update -y
```

```
# Install Apache (httpd)
```

```
sudo yum install -y httpd
```

```
# Start the service
```

```
sudo systemctl start httpd
```

```
# Enable it to start automatically on reboot
```

```
sudo systemctl enable httpd
```

```
# Create a simple landing page
```

```
echo "<h1>Welcome to Pratham's EC2 Web Server</h1>" | sudo tee
```

/var/www/html/index.html

Step 4: Allow HTTP Access

You already know how to edit Security Groups!

1. Go to your instance's Security Groups.
2. Edit Inbound Rules.
3. Add a new rule: Type: HTTP | Port: 80 | Source: Anywhere-IPv4 (0.0.0.0/0).
4. Save rules.

Step 5: Verify the Server

1. Copy the Public IPv4 address of your instance (e.g., 100.48.208.231).
2. Paste it into your browser's address bar.
3. You should see your "Welcome" message.

Step 6: Resize the Instance (The "Resource Modification" Part)

This is a two-part task that requires the instance to be Stopped first.

Part A: Changing Instance Type

1. Go to the EC2 Dashboard and Stop your instance.
2. Once stopped, click Actions -> Instance Settings -> Change instance type.
3. Change it from t2.micro to t3.micro (or any other type allowed in your lab).
4. Click Apply.

Part B: Modifying EBS Storage

1. In the left menu, go to Elastic Block Store -> Volumes.
2. Select the volume attached to your instance.
3. Click Actions -> Modify volume.
4. Change the Size (e.g., if it's 8 GiB, change it to 10 GiB).
5. Click Modify and confirm.
6. Go back to Instances and Start your instance again.

CLOUDTRAIL

This is an essential task for Auditing and Security. CloudTrail acts like a "CCTV camera" for your AWS account, recording every click, command, and API call made by any user or service.

Step 1 & 2: Open CloudTrail and Create a Trail

1. Search for CloudTrail in the AWS Management Console.
2. Click Create trail.
3. Trail name: Pratham-Activity-Log.
4. Storage location: Select Create new S3 bucket.
5. Trail Log Bucket and Folder: AWS will suggest a unique name like awscloudtrail-logs-213348783129-xxxx. Leave it as is.
6. Apply trail to all regions: Ensure this is set to Yes. This ensures that even if someone launches an instance in London or Tokyo, it gets logged.
7. Click Next, keep "Management events" checked, and click Next again to Create trail.

Step 3: The S3 Bucket (Verification)

AWS actually creates the S3 bucket for you during Step 2.

1. Go to the S3 Console.

2. Search for the bucket name you just saw in the CloudTrail setup.

3. Open the bucket. You will see a folder structure like AWSLogs/AccountID/CloudTrail/.

o Note: It may be empty for a few minutes; logs take a bit of time to "deliver" to S3.

Step 4: Perform an AWS Action (The Trigger)

To test if it's working, we need to create a "paper trail":

1. Go to the EC2 Dashboard.

2. Select your VPC-Web-Server (or any instance).

3. Click Instance state -> Stop instance.

4. Once it stops, click Instance state -> Start instance.

Step 5: Verify the Log (The Evidence)

CloudTrail has two places to check: the Event History (fastest) and the S3 Bucket (permanent storage).

Checking Event History (Immediate):

1. Go back to the CloudTrail Dashboard.

2. Click Event history in the left menu.

3. Look for events like:

o StopInstances

o StartInstances

4. Click on the event name. You will see exactly who did it (your username), when, and from which IP address.

Checking S3 (Formal Storage):

1. Go to your CloudTrail S3 bucket.

2. Navigate deep into the folders: Region > Year > Month > Day.

3. You will find .json.gz files. These are the raw logs containing the same data you saw in Event History.

VPC

STEP 1 — Login AWS

Open browser

Go to

AWS Console

Login

STEP 2 — Open VPC Dashboard

Search:

VPC

Open

Amazon VPC

STEP 3 — Create VPC

Click:

Create VPC

Select:

VPC Only

Enter:

Field	Value
-------	-------

Name	MyVPC
------	-------

IPv4 CIDR	10.0.0.0/16
-----------	-------------

Click:

Create VPC

STEP 4 — Create Public Subnet

Go to:

Subnets

Click:

Create Subnet

Enter:

Field	Value
-------	-------

VPC	MyVPC
-----	-------

Name	Public-Subnet
------	---------------

AZ	ap-south-1a
----	-------------

CIDR	10.0.1.0/24
------	-------------

Click:

Create

STEP 5 — Create Private Subnet

Again click:

Create Subnet

Enter:

Field	Value
-------	-------

VPC	MyVPC
-----	-------

Name	Private-Subnet
------	----------------

AZ	ap-south-1a
----	-------------

CIDR	10.0.2.0/24
------	-------------

Click:

Create

STEP 6 — Create Another Public Subnet (2nd AZ)

Field	Value
-------	-------

Name	Public-Subnet-2
------	-----------------

AZ	ap-south-1b
----	-------------

CIDR	10.0.3.0/24
------	-------------

Create subnet.

STEP 7 — Create Another Private Subnet (2nd AZ)

Field	Value
-------	-------

Name	Private-Subnet-2
------	------------------

AZ	ap-south-1b
----	-------------

CIDR	10.0.4.0/24
------	-------------

Create subnet.

STEP 8 — Create Internet Gateway

Open:

Internet Gateways

Click:

Create Internet Gateway

Name:

My-IGW

Click:

Create

STEP 9 — Attach IGW to VPC

Select IGW

Click:

Actions → Attach to VPC

Select:

MyVPC

Attach

STEP 10 — Create Public Route Table

Open:

Route Tables

Click:

Create Route Table

Enter:

Field	Value
-------	-------

Name	Public-RT
------	-----------

VPC	MyVPC
-----	-------

Create table.

STEP 11 — Add Internet Route

Select:

Public-RT

Open:

Routes → Edit Routes

Add:

Destination	Target
0.0.0.0/0	Internet Gateway

Save

STEP 12 — Associate Public Subnets

Open:

Subnet Associations

Select:

Public-Subnet

Public-Subnet-2

Save

STEP 13 — Create NAT Gateway

Allocate Elastic IP

Open:

Elastic IPs

Click:

Allocate Elastic IP

Create NAT Gateway

Open:

NAT Gateways

Click:

Create NAT Gateway

Enter:

Field	Value
-------	-------

Name	My-NAT
------	--------

Subnet	Public-Subnet
--------	---------------

Elastic IP	Allocated EIP
------------	---------------

Click:

Create

STEP 14 — Create Private Route Table

Create route table:

Private-RT

Add route:

Destination	Target
-------------	--------

0.0.0.0/0	NAT Gateway
-----------	-------------

Associate:

Private-Subnet

Private-Subnet-2

STEP 15 — Create Security Group

Open:

Security Groups

Click:

Create Security Group

Enter:

Field	Value
-------	-------

Name	WebServer-SG
------	--------------

VPC	MyVPC
-----	-------

Add Inbound Rules

Type	Port	Source
------	------	--------

HTTP	80	Anywhere
------	----	----------

SSH	22	My IP
-----	----	-------

Create security group.

STEP 16 — Launch EC2 Instance

Open

Amazon EC2

Click:

Launch Instance

Configure

Field Value

Name WebServer

AMI Amazon Linux 2

Type t2.micro

VPC MyVPC

Subnet Public-Subnet

Auto Public IP Enable

Security Group WebServer-SG

STEP 17 — Add User Data Script

Paste in Advanced Details → User Data

```
#!/bin/bash
yum update -y
yum install httpd -y
systemctl start httpd
systemctl enable httpd
echo "<h1>AWS Web Server Running</h1>" > /var/www/html/index.html
```

Launch instance.

STEP 18 — Verify Web Server

Wait for instance state:

Running

Copy:

Public DNS

Open browser:

http://Public-DNS

OUTPUT

Web page shows:

AWS Web Server Running